

НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ, МЕХАНИКИ И ОПТИКИ

Факультет «инфокоммуникационных технологий»

Направление подготовки «инфокоммуникационные технологии и системы связи»

Лабораторная работа №6
«Работа с сокетами»

Выполнил:

Гаврилов Кирилл Станиславович

Группа К3322

Проверил:

Марченко Елена Вадимовна

Санкт-Петербург
2025

Цель работы: овладеть практическими навыками и умениями реализации web-серверов и использования сокетов.

Ход работы:

Задание 1.

Были созданы два файла – серверная и клиентская части. Серверная часть ожидает соединения и сообщения от клиента, после чего анализирует полученное сообщение и отправляет ответ клиенту и закрывает соединение с сокетом. Клиент, в свою очередь, устанавливает соединение с сервером, отправляет ему сообщение и выводит полученный ответ. Реализация сервера представлена на рисунке 1, клиента – на рисунке 2.

```
import socket

sock = socket.socket()
sock.bind(('', 9090))
sock.listen()

conn, addr = sock.accept()

data = conn.recv(1024)
if data == b'Hello, server':
    print(data.decode())
    conn.send(b'Hello, client')
else:
    print('Another data')
    conn.send(b'Who you are?')

conn.close()
```

Рисунок 1 – серверная часть

```
import socket

sock = socket.socket()
sock.connect(('localhost', 9090))

sock.send(b'Hello, server')

data = sock.recv(1024)
print(data.decode())

sock.close()
```

Рисунок 2 – клиентская часть

```
(.venv) kswHoks@MacBook-Air-Kirill-3 task1 % python client.py
Hello, client
```

Рисунок 3 – выполнение файла client.py при запущенном сервере

```
(.venv) kswHoks@MacBook-Air-Kirill-3 task1 % python server.py
Hello, server
```

Рисунок 4 – полученное сервером сообщение от клиента

Задание 2.

Были созданы два файла для имитации клиента и сервера. Сервер ожидает от клиента сообщение с параметрами, на основе которых считается площадь параллелограмма и результат отправляется обратно клиенту. Клиент же запрашивает у пользователя параметры для передачи на сервер, после чего получает ответ и выводит его. Код серверной части представлен на рисунке 5, клиентской – на рисунке 6.

```

import socket
from math import sin, pi

sock = socket.socket()
sock.bind('', 9090)
sock.listen()

conn, addr = sock.accept()

data = conn.recv(1024).decode()
params = list(map(float, data.split()))

if len(params) == 2:
    res = str(params[0] * params[1]).encode()
elif len(params) == 3:
    res = str(params[0] * params[1] * sin(params[2] * pi / 180)).encode()
else:
    res = b'Invalid params'

conn.send(res)

conn.close()

```

Рисунок 5 – серверная часть

```

import socket

sock = socket.socket()
sock.connect(('localhost', 9090))

print('Доступные формулы вычисления площади параллелограмма:\n'
      'Сторона параллелограмма * высота, проведенная к этой стороне\n'
      'Произведение смежных сторон параллелограмма на синус угла между ними (ввести угол в градусах)')
data = input().encode()

sock.send(data)

res = sock.recv(1024)
print(res.decode())

sock.close()

```

Рисунок 6 – клиентская часть

```
(.venv) kswhoks@MacBook-Air-Kirill-3 task2 % python client.py
Доступные формулы вычисления площади параллелограмма:
Сторона параллелограмма * высота, проведенная к этой стороне
Произведение смежных сторон параллелограмма на синус угла между ними (ввести угол в градусах)
4 5
20.0
```

Рисунок 7 – вывод клиентской части

Задание 3.

Был создан файл для реализации серверной части приложения, а также файл index.html, содержащий разметку необходимой страницы. Сервер ожидает подключения, получает сообщение от клиента, после чего отправляет HTTP-запрос с заголовками и страницей index.html. Реализация сервера представлена на рисунке 8, разметка страницы – на рисунке 9.

```
import socket
import sys

sock = socket.socket()
sock.bind('', 9090)
sock.listen(5)

conn, addr = sock.accept()

path = sys.path[0] + '/index.html'
headers = 'HTTP/1.1 200 OK\r\nContent-Type: text/html; charset=utf-8\r\n\r\n'

inp = conn.recv(1024).decode()
print(inp)
with open(path, 'r', encoding='utf-8') as file:
    data = headers.encode() + ''.join(file.readlines()).encode()
    conn.send(data)

conn.close()
```

Рисунок 8 – реализация сервера

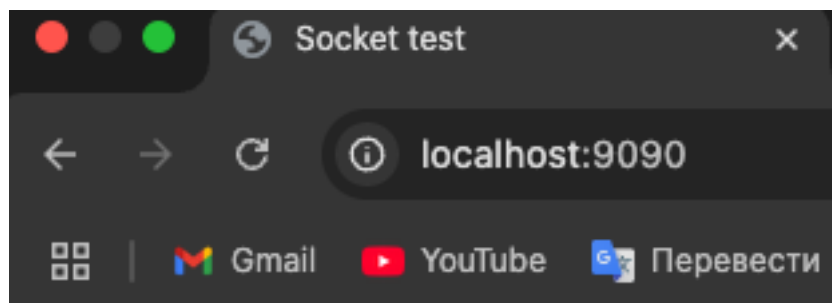
```

<!DOCTYPE html>
<html>
<head>
    <title>Socket test</title>
    <meta charset="utf-8">
</head>
<body>
    <h1>Sockets</h1>
</body>
</html>

```

Рисунок 9 – разметка страницы index.html

При запущенном сервере в браузере переходим по ссылке localhost:9090, где отобразится страница index.html. Сервер, в свою очередь, получил заголовки запроса от браузера и вывел их в консоль.



Sockets

Рисунок 10 – страница index.html

Задание 4.

Был реализован многопользовательский чат на основе сокетов и многопоточности. В серверной части в бесконечном цикле ожидается подключение клиента и запускается поток, ожидающий от него сообщения. В клиентской части реализованы два потока: для приема сообщений и для их отправки. Реализация серверной и клиентской частей представлена на рисунках 11-13.

```

server = socket.socket()
server.bind(('', 9090))
server.listen(5)

clients = list()
has_had_connection = False

while True:

    client_socket, address = server.accept()
    clients.append(client_socket)

    cli_name = client_socket.recv(1024).decode()

    client_thread = threading.Thread(target=client_handler, args=(client_socket, address, cli_name))
    client_thread.start()

```

Рисунок 11 – серверная часть: основной код

```

1 usage
def client_handler(sock, adr, name):
    print(f'New client: {name} {adr}')
    global has_had_connection
    has_had_connection = True
    while True:
        message = sock.recv(1024)
        if not message or message == b'exit':
            break
        message = message.decode()
        print(f'Message from {name}: "{message}"')
        for cli in clients:
            if cli != sock:
                cli.send(f'{name}: "{message}"'.encode())
    sock.close()
    print(f'Connection with client {name} {adr} has closed')
    clients.remove(sock)

```

Рисунок 12 – серверная часть: обработчик для клиента

```

1 usage
def get_message(sock):
    while True:
        message = sock.recv(1024)
        if not message:
            break
        print(message.decode())

1 usage
def send_message(sock):
    while True:
        message = input()
        if message == 'exit':
            sock.send('exit'.encode())
            break
        else:
            sock.send(message.encode())

client = socket.socket()
client.connect(('localhost', 9090))

client_name = input("Enter your name: ")
client.send(client_name.encode())

get_thread = threading.Thread(target=get_message, args=(client,))
get_thread.start()

send_thread = threading.Thread(target=send_message, args=(client,))
send_thread.start()

```

Рисунок 13 – клиентская часть


```
(.venv) kswboks@MacBook-Air-Kirill-3 task4 % python server.py
New client: Kirill ('127.0.0.1', 49729)
New client: Timoha ('127.0.0.1', 49730)
New client: Serezha ('127.0.0.1', 49731)
Message from Kirill: "Hello"
Message from Timoha: "Hi"
Message from Serezha: "Aloha"
Message from Kirill: "52"
Message from Timoha: "yes yes 53"
Message from Serezha: "no no 52"
```

```
(.venv) kswboks@MacBook-Air-Kirill-3 task4 % python server.py
New client: Kirill ('127.0.0.1', 49729)
New client: Timoha ('127.0.0.1', 49730)
New client: Serezha ('127.0.0.1', 49731)
Message from Kirill: "Hello"
Message from Timoha: "Hi"
Message from Serezha: "Aloha"
Message from Kirill: "52"
Message from Timoha: "yes yes 53"
Message from Serezha: "no no 52"
```

Рисунок 14.1 – пример лога на сервере

```
(.venv) kswboks@MacBook-Air-Kirill-3 task4 % python server.py
New client: ('127.0.0.1', 49759)
Connection with client ('127.0.0.1', 49759) has closed
New client: Kirill ('127.0.0.1', 49761)
New client: Timoha ('127.0.0.1', 49762)
Message from Timoha: "hi"
Connection with client Kirill ('127.0.0.1', 49761) has closed
Connection with client Timoha ('127.0.0.1', 49762) has closed
```

Рисунок 14.2 – пример лога на сервере

```
(.venv) kswhoks@MacBook-Air-Kirill-3 task4 % python client.py
Enter your name: Kirill
Hello
Timoha: "Hi"
Serezha: "Aloha"
52
Timoha: "yes yes 53"
Serezha: "no no 52"
```

Рисунок 15 – пример от лица пользователя Kirill

Вывод: в ходе работы были изучены принципы реализации web-серверов с применением сокетов, а также затронута тема многопоточности.